
Ramulator2.0 Basics

1st, Mar. 2026 Minsung Kim

***Intelligent System
Laboratory***

Contents

- 1. Ramulator2.0의 기본적 특징**
- 2. Host**
- 3. Memory Subsystem**
- 4. Memory Controller**
- 5. DDR Operation**

Ramulator2.0의 기본적 특징

❑ Cycle Accurate Simulator

- Ramulator2.0은 Functional한 동작을 simulate하지 않고, Cycle by Cycle 동작만을 Simulation한다.
- Functional simulation
 - Executes programs correctly. Usually no timing information
 - Used to validate correctness of compilers, etc.
 - RISC-V Spike, QEMU, gem5 "atomic" mode
- Instrumentation-based
 - Often binary translation. Runs on actual hardware with callbacks
 - Like trace-based. Not flexible to new ISA. Some things opaque
 - PIN, NVBit
- Trace-based simulation
 - Generate addresses/events and re-execute
 - Can be fast (no need to do functional simulation). Reuse traces
 - If execution depends on timing, this will not work!
 - "Specialized" simulators for single aspect (e.g., just cache hit/miss)

	Development time	Evaluation time	Accuracy	Coverage
functional simulation	excellent	good	poor	poor
instrumentation	excellent	very good	poor	poor
specialized cache and predictor simulation	good	good	good	limited
full trace-driven simulation	poor	poor	very good	excellent
full execution-driven simulation	very poor	very poor	excellent	excellent

Ramulator2.0의 기본적 특징

□ Trace Based Simulator

- Ramulator2.0은 Functional한 동작을 simulate하지 않고, Cycle by Cycle 동작만을 Simulation한다.
- Ramulator2.0은 Input을 Trace로 넣고, 해당 Trace를 수행한 결과를 Report로 받는다.
- Trace file은 내가 진행하고 싶은 시나리오를 설정한 파일이라고 생각하면 된다.

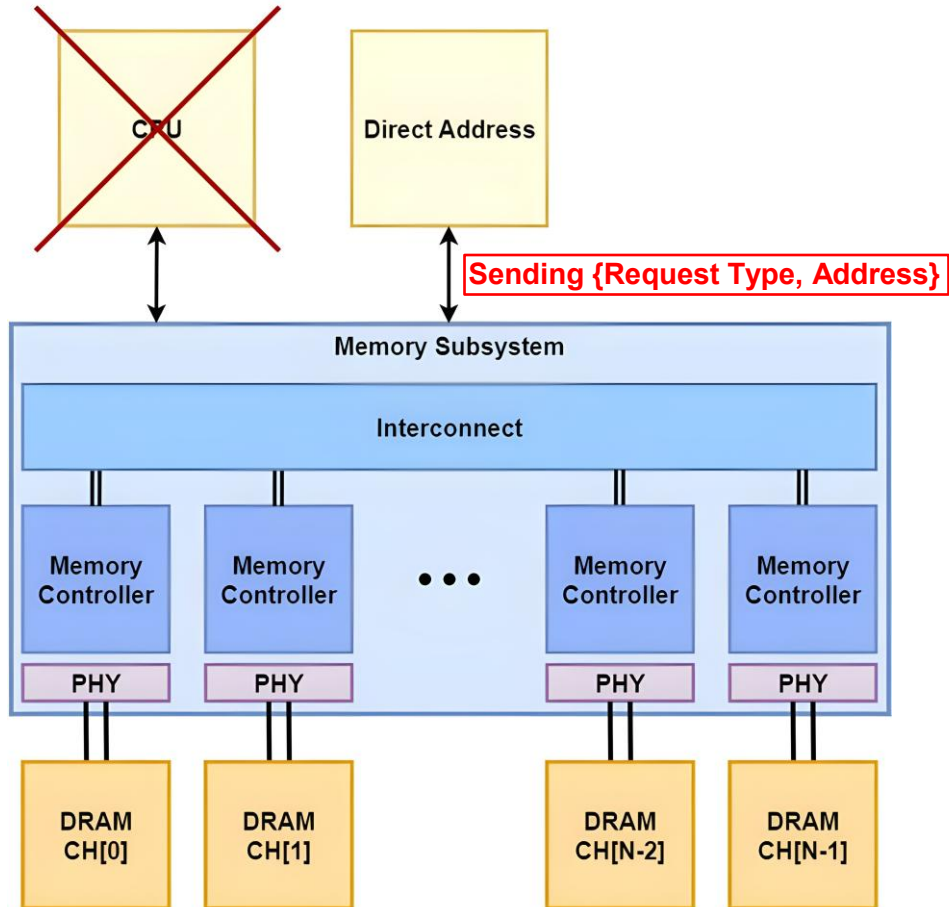
```
1. LD 0x00000000
2. BU 5
3. ST 0x00000400
4. LD 0x00000800
5. ST 0x00000C00
6. BU 5
7. LD 0x00001000
8. ST 0x00001400
9. LD 0x00001800
10. ST 0x00001C00
```

Trace의 예시

```
1. Frontend:
2.   impl: LoadStoreTrace
3.
4. MemorySystem:
5.   impl: GenericDRAM
6.   total_num_other_requests: 0
7.   total_num_write_requests: 4
8.   total_num_read_requests: 4
9.   memory_system_cycles: 144
10.
11. DRAM:
12.   impl: DDR4
13.   active_cycles_rank1: 0
14.   pre_background_energy_rank1: 8.096759999999997
15.   total_background_energy_rank0: 8.6515379999999986
16.   idle_cycles_rank1: 144
17.   total_cmd_energy: 3.5885639999999999
18.   total_cmd_energy_rank0: 3.5885639999999999
19.   total_energy_rank0: 12.240101999999998
20.   act_background_energy_rank0: 6.7960304999999996
21.   pre_background_energy_rank0: 1.8555074999999999
22.   active_cycles_rank0: 111
23.   act_background_energy_rank1: 0
24.   total_energy: 20.336861999999996
25.   idle_cycles_rank0: 33
26.   total_energy_rank1: 8.096759999999997
27.   total_background_energy_rank1: 8.096759999999997
28.   total_background_energy: 16.748297999999998
29.   total_cmd_energy_rank1: 0
30. AddrMapper:
31.   impl: RoBaRaCoCh
32.
33. Controller:
34.   impl: Generic
35.   id: Channel 0
36.   avg_read_latency_0: 36.75
37.   read_queue_len_avg_0: 0.909722209
38.   write_queue_len_0: 278
39.   queue_len_0: 452
40.   num_other_reqs_0: 0
41.   num_write_reqs_0: 4
42.   read_latency_0: 147
43.   priority_queue_len_avg_0: 0.298611104
44.   row_hits_0: 6
45.   priority_queue_len_0: 43
46.   row_misses_0: 2
47.   row_conflicts_0: 0
48.   read_row_misses_0: 1
49.   queue_len_avg_0: 3.13888884
50.   read_row_conflicts_core_0: 0
51.   read_row_hits_0: 3
52.   write_queue_len_avg_0: 1.93055558
53.   read_row_conflicts_0: 0
54.   write_row_misses_0: 1
55.   write_row_conflicts_0: 0
56.   read_queue_len_0: 131
57.   write_row_hits_0: 3
58.   read_row_hits_core_0: 0
59.   read_row_misses_core_0: 0
60.   num_read_reqs_0: 4
61. Scheduler:
62.   impl: FRFCFS
63. RefreshManager:
64.   impl: AllBank
65.
66.
67.
68. RowPolicy:
69.   impl: ClosedRowPolicy
70.   num_close_reqs: 2
```

Report의 예시

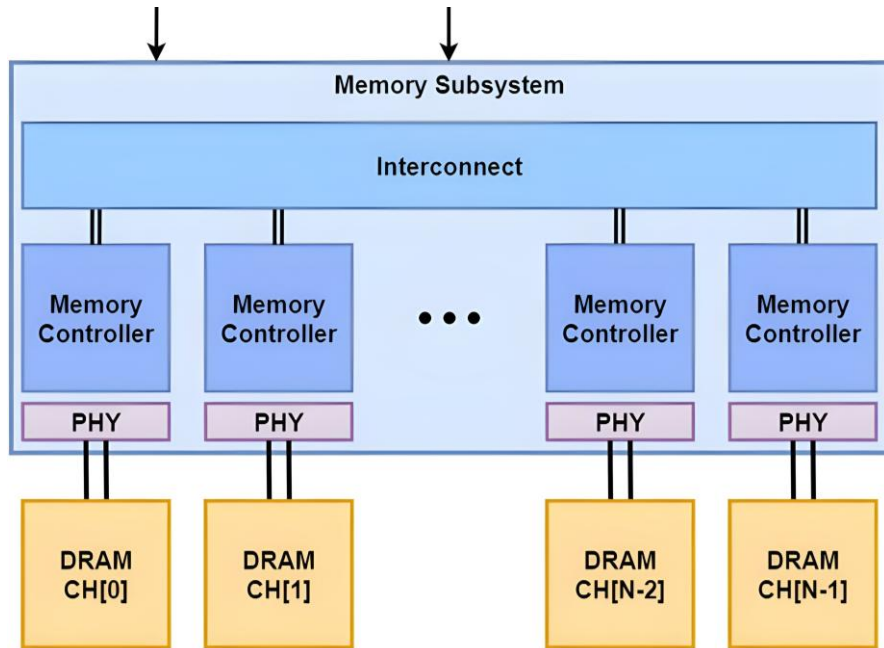
□ Host (ex. CPU-Cache)



- Ramulator2.0은 추상적인 CPU + Cache Modeling이 존재
- 기존의 CPU + Cache Modeling을 배제하고, Memory Access를 Direct하게 할 수 있도록 Host를 구성한다.
- Direct Address 동작을 모델링하게 되면 다음과 같이 동작한다.
 1. Trace file을 읽고, Parsing을 진행한다.
 - Parsing 후 {Request Type, Address}로 분리된다.
 2. Request Type에 따라, Read인지 Write인지 구분하여 Address와 함께 Memory System으로 보낸다.

Memory System

□ Memory Subsystem

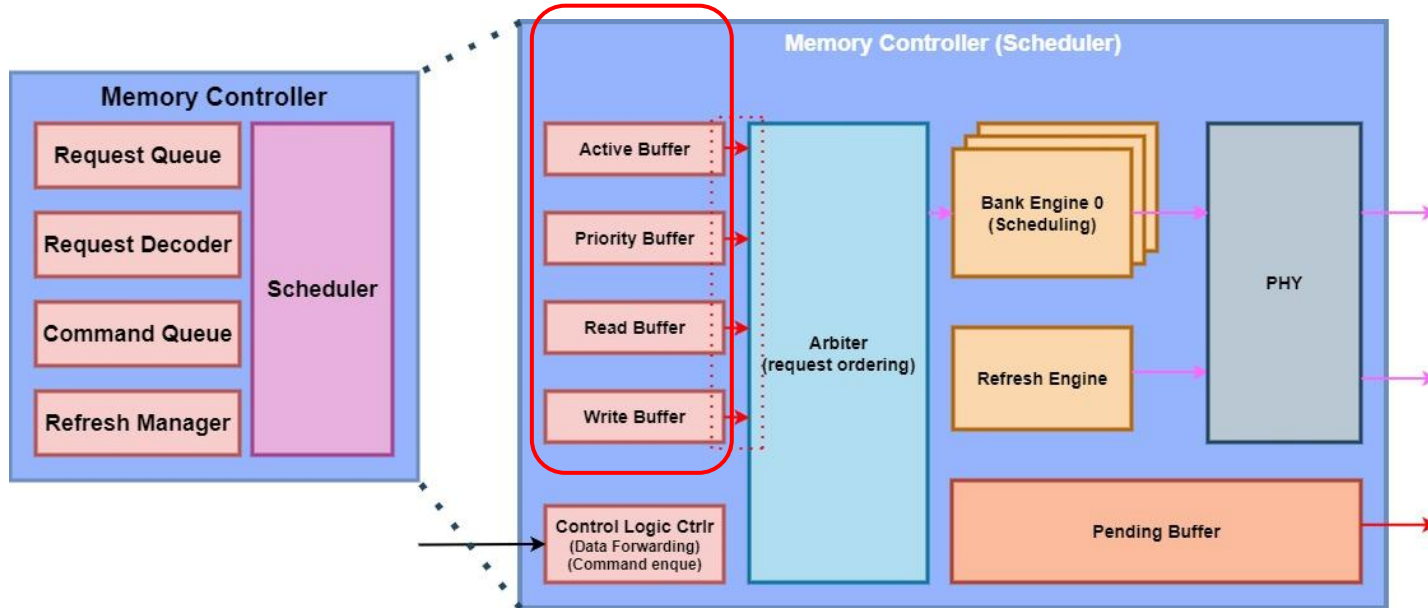


- Host와 통신하기 위해, DIMM과 Host사이에는 Memory Subsystem이 존재
- Memory Subsystem을 모델링 시 다음과 같이 동작한다
 1. Initialization
 2. Address/Command Translation
 - Host의 read/write 요청 → DRAM Vector
 3. DRAM Vector의 Channel id를 추출
 4. 해당 Channel의 Memory Controller에게 Request를 send

Memory System

□ Memory Controller – based on Ramulator/DRAM-Sim Implementations

Simulation Report에 queue_len이라는 지표는 해당 Buffer에 Request가 얼마나 있었는가를 통계 낸 것이다



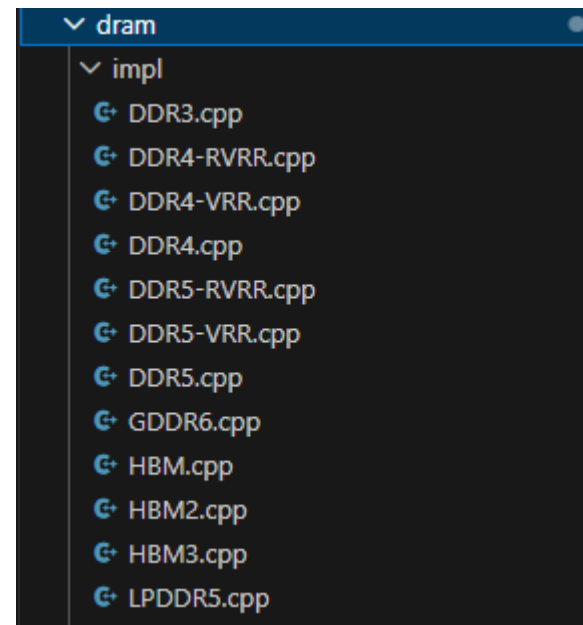
Memory Controller를 모델링 시 다음과 같이 동작한다

1. Request Decode
 - Read/Write 여부를 Decode
 - 적절한 Buffer에 넣음
2. Command Scheduling
 - 직전에 ACT command를 발행했는지
 - Refresh Command인지
 - 일반 RD/WR command인지
 - Timing Constraints를 만족하는지
 - 위 사항들을 check하여 Scheduling
3. Refresh Managing
 - 일정 fixed된 주기마다 Refresh 발행

DRAM Chip

□모델링 된 DRAM들 List

- DDR3.cpp
- DDR4-RVRR.cpp
- DDR4-VRR.cpp
- DDR4.cpp
- DDR5-RVRR.cpp
- DDR5-VRR.cpp
- DDR5.cpp
- GDDR6.cpp
- HBM.cpp
- HBM2.cpp
- HBM3.cpp
- LPDDR5.cpp



□ DRAM의 동작 모델링

- 앞서 언급했듯이, Ramulator2.0은 DRAM의 Timing만 모델링하고, 실제 동작은 모델링하지 않는다.
- 이는 실제 Data를 저장/전송하는 동작을 수행하지 않는다는 것이다.
- Data의 경우 있다고 가정하고, Address만 가지고 Cycle by Cycle Simulation이 진행된다.
- **DRAM 모델들은 크게 3가지 동작을 수행한다.**
 1. Timing Constraints를 참고하여 각 Command가 발행될 수 있는 Timing을 update
 - ex. ACT 후, ACT가 다시 issue되려면 → 다음 ACT는 (현재 cycle + tRRDS)이후에 issue될 수 있음
 2. 각 Command가 실행된 횟수를 기록하여, 나중에 Power계산에 사용
 3. DRAM state를 Update
 - State의 경우 보통 Opened, Closed, Refreshing으로 구성되어 있음
 - Opened는 Row가 열린 상태, Closed는 Row가 닫힌 상태, Refreshing은 해당 Bank가 Refresh인 상태
 - ex. ACT command issue 시 → 해당 Bank가 Open인 상태로 전환됨
 - ex. PREA command issue 시 → 모든 Bank가 Closed인 상태로 전환됨

Ramulator2.0 사용법

1st, Mar. 2026 Minsung Kim

***Intelligent System
Laboratory***

Ramulator 2.0 setup

❑ Using the revised version of Ramulator2.0 (load/store enabled)

- Clone the repository (아래 url주소는 추후 바뀔 수 있습니다)
\$ git clone https://github.com/RayKim3103/my_ramulator2
- Configure the project and build the executable
\$ mkdir build
\$ cd build
\$ cmake ..
\$ make -j
\$ cp ./ramulator2 ../ramulator2
\$ cd ..
- **Executing Simulation!!**
\$./ramulator2 -f ./example_config.yaml
\$./ramulator2 -f ./example_config_LDST.yaml

Ramulator 2.0 setup

□ Ramulator2.0에서의 simulation 설정

- Ramulator2.0에서 Simulation은 크게 2가지 File을 넣어서 돌린다.
 - trace file
 - 어떤 Request들을 Memory로 보낼 지에 대한 시나리오를 구성한 파일
 - yaml file
 - 어떤 Host를 사용할 것인지
 - 어떤 Address Translation(Physical ↔ Virtual Address)을 사용할 것인지
 - 어떤 Memory System을 사용할 것인지
 - 어떤 Memory Controller를 사용할 것인지
 - 어떤 DDR을 사용할 것인지
- 위 spec을 결정하는 파일이 yaml file이다.

Ramulator 2.0 setup

❑ Ramulator2.0에서의 simulation 설정 – trace file

- memory-trace parser model – Host: Direct Address 동작에 해당
 - 입력은 (LD/ST) (physical load/store address) 형식으로 줌
 - 입력에서 LD/ST 중간에 n Cycle동안 Bubble을 주고 싶다면, (BU) (stalled_cycles) 형식으로 줌
 - Simulation은 Frontend(=Host)가 instruction을 매 cycle마다 1줄씩 trace를 memory system으로 send

관련된 내용은 아래 링크의 2page 끝에서 3page 중간까지 적혀 있습니다. (Onur Mutlu 강의)

[fetch.php](#)

```
example_LDST.trace
1 LD 0x00000000
2 BU 5
3 ST 0x00000400
4 LD 0x00000800
5 ST 0x00000C00
6 BU 5
7 LD 0x00001000
8 ST 0x00001400
9 LD 0x00001800
10 ST 0x00001C00
11
```

Ramulator 2.0 setup

❑ Ramulator2.0에서의 simulation 설정 – trace file

- memory-trace parser model
 - Physical Address에서 DRAM vector로의 변환은 아래 folder에 정의되어 있음
 - src > addr_mapper > impl > linear_mappers.cpp

▪ example. BU 명령을 넣지 않은 Trace

• Cycle by Cycle 분석

동작 예시 1

```
≡ example_LDST.trace
1  LD 0x00000000
2  ST 0x00000400
3  LD 0x00000800
4  ST 0x00000C00
5  LD 0x00001000
6  ST 0x00001400
7  LD 0x00001800
8  ST 0x00001C00
9
```



```
cycle: 0 System - send() Read Request Address: 0 / Request Address Vector: 0 0 0 0 0 0
cycle: 0 Controller - send() Read - Request Address Vector: 0 0 0 0 0 0
cycle: 1 System - send() Write Request Address: 1024 / Request Address Vector: 0 0 0 0 0 16
cycle: 1 Controller - send() Write - Request Address Vector: 0 0 0 0 0 16
cycle: 2 System - send() Read Request Address: 2048 / Request Address Vector: 0 0 0 0 0 32
cycle: 2 Controller - send() Read - Request Address Vector: 0 0 0 0 0 32
cycle: 3 System - send() Write Request Address: 3072 / Request Address Vector: 0 0 0 0 0 48
cycle: 3 Controller - send() Write - Request Address Vector: 0 0 0 0 0 48
cycle: 4 System - send() Read Request Address: 4096 / Request Address Vector: 0 0 0 0 0 64
cycle: 4 Controller - send() Read - Request Address Vector: 0 0 0 0 0 64
cycle: 5 System - send() Write Request Address: 5120 / Request Address Vector: 0 0 0 0 0 80
cycle: 5 Controller - send() Write - Request Address Vector: 0 0 0 0 0 80
cycle: 6 System - send() Read Request Address: 6144 / Request Address Vector: 0 0 0 0 0 96
cycle: 6 Controller - send() Read - Request Address Vector: 0 0 0 0 0 96
cycle: 7 System - send() Write Request Address: 7168 / Request Address Vector: 0 0 0 0 0 112
cycle: 7 Controller - send() Write - Request Address Vector: 0 0 0 0 0 112
```

Ramulator 2.0 setup

❑ Ramulator2.0에서의 simulation 설정 – trace file

- memory-trace parser model
 - example. BU 명령을 넣은 Trace

동작 예시 2

```
≡ example_LDST.trace
1  LD 0x00000000
2  BU 5
3  ST 0x00000400
4  LD 0x00000800
5  ST 0x00000C00
6  BU 5
7  LD 0x00001000
8  ST 0x00001400
9  LD 0x00001800
10 ST 0x00001C00
11
```



• Cycle by Cycle 분석

```
cycle: 0 System - send() Read Request Address: 0 / Request Address Vector: 0 0 0 0 0
cycle: 0 Controller - send() Read - Request Address Vector: 0 0 0 0 0
cycle: 5 Bubble finished
cycle: 6 System - send() Write Request Address: 1024 / Request Address Vector: 0 0 0 0 16
cycle: 6 Controller - send() Write - Request Address Vector: 0 0 0 0 16
cycle: 7 System - send() Read Request Address: 2048 / Request Address Vector: 0 0 0 0 32
cycle: 7 Controller - send() Read - Request Address Vector: 0 0 0 0 32
cycle: 8 System - send() Write Request Address: 3072 / Request Address Vector: 0 0 0 0 48
cycle: 8 Controller - send() Write - Request Address Vector: 0 0 0 0 48
cycle: 13 Bubble finished
cycle: 14 System - send() Read Request Address: 4096 / Request Address Vector: 0 0 0 0 64
cycle: 14 Controller - send() Read - Request Address Vector: 0 0 0 0 64
cycle: 15 System - send() Write Request Address: 5120 / Request Address Vector: 0 0 0 0 80
cycle: 15 Controller - send() Write - Request Address Vector: 0 0 0 0 80
cycle: 16 System - send() Read Request Address: 6144 / Request Address Vector: 0 0 0 0 96
cycle: 16 Controller - send() Read - Request Address Vector: 0 0 0 0 96
```

Ramulator 2.0 setup

□ Ramulator2.0에서의 simulation 설정 – **yaml file**

- yaml file (해당 yaml은 CPU모델링을 사용할 때의 yaml입니다. Direct Address Host를 쓴 yaml은 [참고](#))

```
1 Frontend:
2   impl: SimpleO3
3   clock_ratio: 8
4   num_expected_insts: 500000
5   traces:
6     - example_inst.trace
7
8   Translation:
9     impl: RandomTranslation
10    max_addr: 2147483648
11
12
13 MemorySystem:
14   impl: GenericDRAM
15   clock_ratio: 3
16
17   DRAM:
18     impl: DDR4
19     org:
20       preset: DDR4_8Gb_x8
21       channel: 1
22       rank: 2
23     timing:
24       preset: DDR4_2400R
25
26   Controller:
27     impl: Generic
28     Scheduler:
29       impl: FRFCFS
30     RefreshManager:
31       impl: AllBank
32     RowPolicy:
33       impl: ClosedRowPolicy
34       cap: 4
35     plugins:
36
37   AddrMapper:
38     impl: RoBaRaCoCh
```

1. Frontend Interface(= Host) 부분

- Trace file에서 Instruction읽고, Memory Request 생성
- Impl: SimpleO3
⇒ Simple Out-of-Order (O3) CPU
- Clock ratio: 8
⇒ Reference CLK 대비 Host CLK 속도
- Num expected insts: 500000
⇒ Simulation이 해당 instruction 수에 도달 시 종료
- Traces
⇒ Instruction trace file(include memory access Inst)
- Impl: RandomTranslation
⇒ Physical Memory ↔ Virtual Memory 변환
⇒ System의 Page Table 등을 간단히 Modeling
- Max addr: 2147483648
⇒ Translation 시 address overflow 방지

Ramulator 2.0 setup

❑ Ramulator2.0에서의 simulation 설정 – **yaml file**

- yaml file (해당 yaml은 CPU모델링을 사용할 때의 yaml입니다. Direct Address Host를 쓴 yaml은 [참고](#))

```
1 Frontend:
2   impl: SimpleO3
3   clock_ratio: 8
4   num_expected_insts: 500000
5   traces:
6     - example_inst.trace
7
8   Translation:
9     impl: RandomTranslation
10    max_addr: 2147483648
11
12
13 MemorySystem:
14   impl: GenericDRAM
15   clock_ratio: 3
16
17   DRAM:
18     impl: DDR4
19     org:
20       preset: DDR4_8Gb_x8
21       channel: 1
22       rank: 2
23     timing:
24       preset: DDR4_2400R
25
26   Controller:
27     impl: Generic
28     Scheduler:
29       impl: FRFCFS
30     RefreshManager:
31       impl: AllBank
32     RowPolicy:
33       impl: ClosedRowPolicy
34       cap: 4
35     plugins:
36
37   AddrMapper:
38     impl: RoBaRaCoCh
```

2. MemorySystem Interface 부분

- Frontend의 Request를 받아 DRAM Ctrlr을 통해 처리
- Latency, en/dequeue, Timing Constraints 처리
- impl: GenericDRAM
 - ⇒ 기본 DRAM 기반 System, Ctrlr와 DRAM을 통합
- clock_ratio: 3
 - ⇒ global CLK 대비 MemorySystem CLK 속도
 - ⇒ 현재: DRAM이 CPU보다 느린 System (= 3:8)

Ramulator 2.0 setup

□ Ramulator2.0에서의 simulation 설정 – **yaml file**

- yaml file (해당 yaml은 CPU모델링을 사용할 때의 yaml입니다. Direct Address Host를 쓴 yaml은 [참고](#))

```
1 Frontend:
2   impl: SimpleO3
3   clock_ratio: 8
4   num_expected_insts: 500000
5   traces:
6     - example_inst.trace
7
8   Translation:
9     impl: RandomTranslation
10    max_addr: 2147483648
11
12
13 MemorySystem:
14   impl: GenericDRAM
15   clock_ratio: 3
16
17   DRAM:
18     impl: DDR4
19     org:
20       preset: DDR4_8Gb_x8
21       channel: 1
22       rank: 2
23     timing:
24       preset: DDR4_2400R
25
26   Controller:
27     impl: Generic
28     Scheduler:
29       impl: FRFCFS
30     RefreshManager:
31       impl: AllBank
32     RowPolicy:
33       impl: ClosedRowPolicy
34       cap: 4
35     plugins:
36
37   AddrMapper:
38     impl: RoBaRaCoCh
```

2. MemorySystem Interface 부분

- **DRAM Section**
- impl: DDR4
⇒ tick() 시 Timing Check / Command Issue 실행.
- org: DDR4_8Gb_x8
⇒ 현재 DRAM preset - 8Gb 용량, x8bit data bus
⇒ Channel/Rank 설정 시 기본 Preset설정을 Override 함
- timing: DDR4_2400R
⇒ Timing preset - nRCD등의 Timing Constraint 정의
⇒ 이를 이용해 tick() 시 Latency 계산

Ramulator 2.0 setup

□ Ramulator2.0에서의 simulation 설정 – **yaml file**

- yaml file

```
1  Frontend:
2    impl: SimpleO3
3    clock_ratio: 8
4    num_expected_insts: 500000
5  traces:
6    - example_inst.trace
7
8  Translation:
9    impl: RandomTranslation
10   max_addr: 2147483648
11
12
13 MemorySystem:
14   impl: GenericDRAM
15   clock_ratio: 3
16
17   DRAM:
18     impl: DDR4
19     org:
20       preset: DDR4_8Gb_x8
21       channel: 1
22       rank: 2
23     timing:
24       preset: DDR4_2400R
25
26   Controller:
27     impl: Generic
28     Scheduler:
29       impl: FRFCFS
30     RefreshManager:
31       impl: AllBank
32     RowPolicy:
33       impl: ClosedRowPolicy
34       cap: 4
35     plugins:
36
37   AddrMapper:
38     impl: RoBaRaCoCh
```

2. MemorySystem Interface 부분

- **Controller Section**
- impl: Generic
 - ⇒ Generic - 기본 Ctrlr
 - ⇒ Request Queue/Scheduling 등 관리
- Scheduler - impl: FRFCFS
 - ⇒ FRFCFS - First-Ready First-Come-First-Serve
 - ⇒ 준비된 Request를 Queue에서 꺼내 우선 처리
- RefreshManager - impl: AllBank
 - ⇒ AllBank - 모든 Bank simultaneous Refresh
- RowPolicy - impl: ClosedRowPolicy
 - ⇒ ClosedRowPolicy - 사용 후 Row 즉시 닫음(Precharge)
 - ⇒ cap:4 - 열려있는 Row 최대 수 제한
- plugins
 - ⇒ 현재 Ramulator에서는 Row Hammering 완화 기법을 plugin으로 제공해줌

Ramulator 2.0 setup

□ Ramulator2.0에서의 simulation 설정 – **yaml file**

- yaml file (해당 yaml은 CPU모델링을 사용할 때의 yaml입니다. Direct Address Host를 쓴 yaml은 [참고](#))

```
1 Frontend:
2   impl: SimpleO3
3   clock_ratio: 8
4   num_expected_insts: 500000
5   traces:
6     - example_inst.trace
7
8   Translation:
9     impl: RandomTranslation
10    max_addr: 2147483648
11
12
13 MemorySystem:
14   impl: GenericDRAM
15   clock_ratio: 3
16
17   DRAM:
18     impl: DDR4
19     org:
20       preset: DDR4_8Gb_x8
21       channel: 1
22       rank: 2
23     timing:
24       preset: DDR4_2400R
25
26   Controller:
27     impl: Generic
28     Scheduler:
29       impl: FRFCFS
30     RefreshManager:
31       impl: AllBank
32     RowPolicy:
33       impl: ClosedRowPolicy
34       cap: 4
35     plugins:
36
37   AddrMapper:
38     impl: RoBaRaCoCh
```

2. MemorySystem Interface 부분

- AddrMapper Section
- impl: RoBaRaCoCh
 - ⇒ Row-Bank-Rank-Column Mapping Scheme
 - ⇒ Requested Address 변환(Physical → DRAM Vector)

[Physical → DRAM Vector Example]

⇒ Physical Address: 0x12345678

⇒ DRAM Vector:

[Channel:0, Rank:1, Bank:2, Row:128, Column:512]

Ramulator 2.0 setup

□ 전체 Report

example_LDST.trace

```
1 LD 0x00000000
2 BU 5
3 ST 0x00000400
4 LD 0x00000800
5 ST 0x00000C00
6 BU 5
7 LD 0x00001000
8 ST 0x00001400
9 LD 0x00001800
10 ST 0x00001C00
11
```

```
! example_config_LDST.yaml
1 Frontend:
2   impl: LoadStoreTrace
3   path: example_LDST.trace # trace 파일 경로 (상대/절대)
4   clock_ratio: 1
5
6 Translation:
7   impl: RandomTranslation
8   max_addr: 2147483648
9
10
11 MemorySystem:
12   impl: GenericDRAM
13   clock_ratio: 1
14
15 DRAM:
16   impl: DDR4
17   org:
18     preset: DDR4_8Gb_x8
19     channel: 1
20     rank: 2
21   timing:
22     preset: DDR4_2400R
23   drampower_enable: true
24   # power_debug: true # option: debug log
25   voltage:
26     preset: Default # option: voltage preset
27   current:
28     preset: Default # option: current preset
29
30 Controller:
31   impl: Generic
32   Scheduler:
33     impl: FRFCFS
34   RefreshManager:
35     impl: AllBank
36   RowPolicy:
37     impl: ClosedRowPolicy
38     cap: 4
39   plugins:
40
41 AddrMapper:
42   impl: RoBaRaCoCh
43
```



```
Frontend:
  impl: LoadStoreTrace

MemorySystem:
  impl: GenericDRAM
  total_num_other_requests: 0
  total_num_write_requests: 4
  total_num_read_requests: 4
  memory_system_cycles: 144

DRAM:
  impl: DDR4
  active_cycles_rank1: 0
  pre_background_energy_rank1: 8.0967599999999997
  total_background_energy_rank0: 8.6515379999999998
  idle_cycles_rank1: 144
  total_cmd_energy: 3.5885639999999999
  total_cmd_energy_rank0: 3.5885639999999999
  total_energy_rank0: 12.2401019999999998
  act_background_energy_rank0: 6.7960304999999996
  pre_background_energy_rank0: 1.8555074999999999
  active_cycles_rank0: 111
  act_background_energy_rank1: 0
  total_energy: 20.3368619999999996
  idle_cycles_rank0: 33
  total_energy_rank1: 8.0967599999999997
  total_background_energy_rank1: 8.0967599999999997
  total_background_energy: 16.748297999999998
  total_cmd_energy_rank1: 0

AddrMapper:
  impl: RoBaRaCoCh

Controller:
  impl: Generic
  id: Channel 0
  avg_read_latency_0: 36.75
  read_queue_len_avg_0: 0.909722209
  write_queue_len_0: 278
  queue_len_0: 452
  num_other_reqs_0: 0
  num_write_reqs_0: 4
  read_latency_0: 147
  priority_queue_len_avg_0: 0.298611104
  row_hits_0: 6
  priority_queue_len_0: 43
  row_misses_0: 2
  row_conflicts_0: 0
  read_row_misses_0: 1
  queue_len_avg_0: 3.13888884
  read_row_conflicts_core_0: 0
  read_row_hits_0: 3
  write_queue_len_avg_0: 1.93055558
  read_row_conflicts_0: 0
  write_row_misses_0: 1
  write_row_conflicts_0: 0
  read_queue_len_0: 131
  write_row_hits_0: 3
  read_row_hits_core_0: 0
  read_row_misses_core_0: 0
  num_read_reqs_0: 4
  Scheduler:
    impl: FRFCFS
  RefreshManager:
    impl: AllBank

RowPolicy:
  impl: ClosedRowPolicy
  num_close_reqs: 2
```

Example - Report의 Read/Write Buffer Length 계산되는 흐름

□ Read/Write Buffer Length 분석

- 7 Cycle동안의 동작만 분석

example_LDST.trace		
R #0	1	LD 0x00000000
W #1	2	ST 0x00000400
R #2	3	LD 0x00000800
W #3	4	ST 0x00000C00
R #4	5	LD 0x00001000
W #5	6	ST 0x00001400
R #6	7	LD 0x00001800
W #7	8	ST 0x00001C00

```
Frontend:
  impl: LoadStoreTrace

MemorySystem:
  impl: GenericDRAM
  total_num_other_requests: 0
  total_num_write_requests: 4
  total_num_read_requests: 4
  memory_system_cycles: 7
DRAM:
  impl: DDR4
AddrMapper:
  impl: RoBaRaCoCh

Controller:
  impl: Generic
  id: Channel 0
  avg_read_latency_0: 0
  read_queue_len_avg_0: 1.42857146
  write_queue_len_0: 12
  queue_len_0: 22
  num_other_reqs_0: 0
  num_write_reqs_0: 4
  read_latency_0: 0
  priority_queue_len_avg_0: 0
  row_hits_0: 0
  priority_queue_len_0: 0
  row_misses_0: 1
  row_conflicts_0: 0
  read_row_misses_0: 1
  queue_len_avg_0: 3.14285707
  read_row_conflicts_core_0: 0
  read_row_hits_0: 0
  write_queue_len_avg_0: 1.71428573
  read_row_conflicts_0: 0
  write_row_misses_0: 0
  write_row_conflicts_0: 0
  read_queue_len_0: 10
  write_row_hits_0: 0
  read_row_hits_core_0: 0
  read_row_misses_core_0: 0
  num_read_reqs_0: 4
Scheduler:
  impl: FRFCFS
RefreshManager:
  impl: AllBank

RowPolicy:
  impl: ClosedRowPolicy
  num_close_reqs: 0
```

Cycle 0

- Frontend tick():
 - send(R #0)
 - read_buffer.enqueue(#0)
- Controller tick():
 - read_buf=1, write_buf=0, pending=0, priority=0
 - s_read_queue_len += 1
 - s_write_queue_len += 0
 - 여기서 statistics 저장
 - 누적: s_read_queue=1, s_write_queue=0
 - Schedule: R #0, get_preq_command() → ACT 필요
 - issue_command(ACT, [0,0,0,0,0,x])
 - is_opening=true → active_buffer.enqueue(R #0)
 - read_buffer.remove(R #1)
 - read_buf=0, active_buf=1, write_buf=0, pending=0

Example - Report의 Read/Write Buffer Length 계산되는 흐름

□ Read/Write Buffer Length 분석

- 7 Cycle동안의 동작만 분석

example_LDST.trace		
R #0	1	LD 0x00000000
W #1	2	ST 0x00000400
R #2	3	LD 0x00000800
W #3	4	ST 0x00000C00
R #4	5	LD 0x00001000
W #5	6	ST 0x00001400
R #6	7	LD 0x00001800
W #7	8	ST 0x00001C00

```
Frontend:
  impl: LoadStoreTrace

MemorySystem:
  impl: GenericDRAM
  total_num_other_requests: 0
  total_num_write_requests: 4
  total_num_read_requests: 4
  memory_system_cycles: 7
DRAM:
  impl: DDR4
AddrMapper:
  impl: RoBaRaCoCh

Controller:
  impl: Generic
  id: Channel 0
  avg_read_latency_0: 0
  read_queue_len_avg_0: 1.42857146
  write_queue_len_0: 12
  queue_len_0: 22
  num_other_reqs_0: 0
  num_write_reqs_0: 4
  read_latency_0: 0
  priority_queue_len_avg_0: 0
  row_hits_0: 0
  priority_queue_len_0: 0
  row_misses_0: 1
  row_conflicts_0: 0
  read_row_misses_0: 1
  queue_len_avg_0: 3.14285707
  read_row_conflicts_core_0: 0
  read_row_hits_0: 0
  write_queue_len_avg_0: 1.71428573
  read_row_conflicts_0: 0
  write_row_misses_0: 0
  write_row_conflicts_0: 0
  read_queue_len_0: 10
  write_row_hits_0: 0
  read_row_hits_core_0: 0
  read_row_misses_core_0: 0
  num_read_reqs_0: 4
Scheduler:
  impl: FRFCFS
RefreshManager:
  impl: AllBank

RowPolicy:
  impl: ClosedRowPolicy
  num_close_reqs: 0
```

Cycle 1

- Frontend tick():
 - send(W #1)
 - write_buffer.enqueue(#1)
- Controller tick():
 - read_buf=0, write_buf=1, pending=0
 - s_read_queue_len += 0
 - s_write_queue_len += 1
 - 여기서 statistics 저장
 - 누적: s_read_queue=1, s_write_queue=1
 - Schedule: active_buffer에서 #0 선택, command=RD
 - check_ready: nRCD=16이므로,
 - clk(1) < ready_clk(16) → false → 처리 안 됨
 - 이제 read_buffer에서 command 골라야 함
 - But, command없기에, 이번 cycle은 command 실행 X
 - read_buf=0, active_buf=1, write_buf=1, pending=0

Example - Report의 Read/Write Buffer Length 계산되는 흐름

□ Read/Write Buffer Length 분석

- 7 Cycle동안의 동작만 분석

example_LDST.trace		
R #0	1	LD 0x00000000
W #1	2	ST 0x00000400
R #2	3	LD 0x00000800
W #3	4	ST 0x00000C00
R #4	5	LD 0x00001000
W #5	6	ST 0x00001400
R #6	7	LD 0x00001800
W #7	8	ST 0x00001C00

```
Frontend:
  impl: LoadStoreTrace

MemorySystem:
  impl: GenericDRAM
  total_num_other_requests: 0
  total_num_write_requests: 4
  total_num_read_requests: 4
  memory_system_cycles: 7
DRAM:
  impl: DDR4
AddrMapper:
  impl: RoBaRaCoCh

Controller:
  impl: Generic
  id: Channel 0
  avg_read_latency_0: 0
  read_queue_len_avg_0: 1.42857146
  write_queue_len_0: 12
  queue_len_0: 22
  num_other_reqs_0: 0
  num_write_reqs_0: 4
  read_latency_0: 0
  priority_queue_len_avg_0: 0
  row_hits_0: 0
  priority_queue_len_0: 0
  row_misses_0: 1
  row_conflicts_0: 0
  read_row_misses_0: 1
  queue_len_avg_0: 3.14285707
  read_row_conflicts_core_0: 0
  read_row_hits_0: 0
  write_queue_len_avg_0: 1.71428573
  read_row_conflicts_0: 0
  write_row_misses_0: 0
  write_row_conflicts_0: 0
  read_queue_len_0: 10
  write_row_hits_0: 0
  read_row_hits_core_0: 0
  read_row_misses_core_0: 0
  num_read_reqs_0: 4
Scheduler:
  impl: FRFCFS
RefreshManager:
  impl: AllBank

RowPolicy:
  impl: ClosedRowPolicy
  num_close_reqs: 0
```

Cycle 2

- Frontend tick():
 - send(R #2)
 - read_buffer.enqueue(#2)
- Controller tick():
 - read_buf=1, write_buf=1, pending=0**
 - s_read_queue_len += 1
 - s_write_queue_len += 1
 - 여기서 statistics 저장
 - 누적: s_read_queue=2, s_write_queue=2
 - Schedule: active_buffer에서 #1 선택, command=RD
 - check_ready: nRCD=16이므로,
 - clk(2) < ready_clk(16) → false → 처리 안 됨
 - 이제 read_buffer에서 command 골라야 함
 - R #2, get_preq_command() → 애도 nRCD 만족 X
 - command없기에, 이번 cycle은 command 실행 X
 - read_buf=1, active_buf=1, write_buf=1, pending=0**

Example - Report의 Read/Write Buffer Length 계산되는 흐름

□ Read/Write Buffer Length 분석

- 7 Cycle동안의 동작만 분석

example_LDST.trace		
R #0	1	LD 0x00000000
W #1	2	ST 0x00000400
R #2	3	LD 0x00000800
W #3	4	ST 0x00000C00
R #4	5	LD 0x00001000
W #5	6	ST 0x00001400
R #6	7	LD 0x00001800
W #7	8	ST 0x00001C00

```
Frontend:
  impl: LoadStoreTrace

MemorySystem:
  impl: GenericDRAM
  total_num_other_requests: 0
  total_num_write_requests: 4
  total_num_read_requests: 4
  memory_system_cycles: 7
DRAM:
  impl: DDR4
AddrMapper:
  impl: RoBaRaCoCh

Controller:
  impl: Generic
  id: Channel 0
  avg_read_latency_0: 0
  read_queue_len_avg_0: 1.42857146
  write_queue_len_0: 12
  queue_len_0: 22
  num_other_reqs_0: 0
  num_write_reqs_0: 4
  read_latency_0: 0
  priority_queue_len_avg_0: 0
  row_hits_0: 0
  priority_queue_len_0: 0
  row_misses_0: 1
  row_conflicts_0: 0
  read_row_misses_0: 1
  queue_len_avg_0: 3.14285707
  read_row_conflicts_core_0: 0
  read_row_hits_0: 0
  write_queue_len_avg_0: 1.71428573
  read_row_conflicts_0: 0
  write_row_misses_0: 0
  write_row_conflicts_0: 0
  read_queue_len_0: 10
  write_row_hits_0: 0
  read_row_hits_core_0: 0
  read_row_misses_core_0: 0
  num_read_reqs_0: 4
Scheduler:
  impl: FRFCFS
RefreshManager:
  impl: AllBank

RowPolicy:
  impl: ClosedRowPolicy
  num_close_reqs: 0
```

Cycle 3

- Frontend tick():
 - send(W #3)
 - write_buffer.enqueue(#3)
- Controller tick():
 - **read_buf=1, write_buf=2, pending=0**
 - s_read_queue_len += 1
 - s_write_queue_len += 2
 - 여기서 statistics 저장
 - 누적: s_read_queue=3, s_write_queue=4
 - Schedule: active_buffer에서 #1 선택, command=RD
 - check_ready: nRCD=16이므로,
 - $\text{clk}(3) < \text{ready_clk}(16) \rightarrow \text{false} \rightarrow$ 처리 안 됨
 - 이제 read_buffer에서 command 골라야 함
 - R #2, get_preq_command() → 애도 nRCD 만족 X
 - command없기에, 이번 cycle은 command 실행 X
 - **read_buf=1, active_buf=1, write_buf=2, pending=0**

Example - Report의 Read/Write Buffer Length 계산되는 흐름

□ Read/Write Buffer Length 분석

- 7 Cycle동안의 동작만 분석

		example_LDST.trace
R #0	1	LD 0x00000000
W #1	2	ST 0x00000400
R #2	3	LD 0x00000800
W #3	4	ST 0x00000C00
R #4	5	LD 0x00001000
W #5	6	ST 0x00001400
R #6	7	LD 0x00001800
W #7	8	ST 0x00001C00

```
Frontend:
  impl: LoadStoreTrace

MemorySystem:
  impl: GenericDRAM
  total_num_other_requests: 0
  total_num_write_requests: 4
  total_num_read_requests: 4
  memory_system_cycles: 7
DRAM:
  impl: DDR4
AddrMapper:
  impl: RoBaRaCoCh

Controller:
  impl: Generic
  id: Channel 0
  avg_read_latency_0: 0
  read_queue_len_avg_0: 1.42857146
  write_queue_len_0: 12
  queue_len_0: 22
  num_other_reqs_0: 0
  num_write_reqs_0: 4
  read_latency_0: 0
  priority_queue_len_avg_0: 0
  row_hits_0: 0
  priority_queue_len_0: 0
  row_misses_0: 1
  row_conflicts_0: 0
  read_row_misses_0: 1
  queue_len_avg_0: 3.14285707
  read_row_conflicts_core_0: 0
  read_row_hits_0: 0
  write_queue_len_avg_0: 1.71428573
  read_row_conflicts_0: 0
  write_row_misses_0: 0
  write_row_conflicts_0: 0
  read_queue_len_0: 10
  write_row_hits_0: 0
  read_row_hits_core_0: 0
  read_row_misses_core_0: 0
  num_read_reqs_0: 4
Scheduler:
  impl: FRFCFS
RefreshManager:
  impl: AllBank

RowPolicy:
  impl: ClosedRowPolicy
  num_close_reqs: 0
```

Cycle 4

- Frontend tick():
 - send(R #4)
 - read_buffer.enqueue(#4)
- Controller tick():
 - **read_buf=2, write_buf=2, pending=0**
 - s_read_queue_len += 2
 - s_write_queue_len += 2
 - 여기서 statistics 저장
 - 누적: s_read_queue=5, s_write_queue=6
 - Schedule: active_buffer에서 #1 선택, command=RD
 - check_ready: nRCD=16이므로,
 - $\text{clk}(4) < \text{ready_clk}(16) \rightarrow \text{false} \rightarrow$ 처리 안 됨
 - 이제 read_buffer에서 command 골라야 함
 - R #2, get_preq_command() → 애도 nRCD 만족 X
 - R #4, get_preq_command() → 애도 nRCD 만족 X
 - command없기에, 이번 cycle은 command 실행 X
 - **read_buf=2, active_buf=1, write_buf=2, pending=0**

Example - Report의 Read/Write Buffer Length 계산되는 흐름

□ Read/Write Buffer Length 분석

- 7 Cycle동안의 동작만 분석

		example_LDST.trace
R #0	1	LD 0x00000000
W #1	2	ST 0x00000400
R #2	3	LD 0x00000800
W #3	4	ST 0x00000C00
R #4	5	LD 0x00001000
W #5	6	ST 0x00001400
R #6	7	LD 0x00001800
W #7	8	ST 0x00001C00

```
Frontend:
  impl: LoadStoreTrace

MemorySystem:
  impl: GenericDRAM
  total_num_other_requests: 0
  total_num_write_requests: 4
  total_num_read_requests: 4
  memory_system_cycles: 7
DRAM:
  impl: DDR4
AddrMapper:
  impl: RoBaRaCoCh

Controller:
  impl: Generic
  id: Channel 0
  avg_read_latency_0: 0
  read_queue_len_avg_0: 1.42857146
  write_queue_len_0: 12
  queue_len_0: 22
  num_other_reqs_0: 0
  num_write_reqs_0: 4
  read_latency_0: 0
  priority_queue_len_avg_0: 0
  row_hits_0: 0
  priority_queue_len_0: 0
  row_misses_0: 1
  row_conflicts_0: 0
  read_row_misses_0: 1
  queue_len_avg_0: 3.14285707
  read_row_conflicts_core_0: 0
  read_row_hits_0: 0
  write_queue_len_avg_0: 1.71428573
  read_row_conflicts_0: 0
  write_row_misses_0: 0
  write_row_conflicts_0: 0
  read_queue_len_0: 10
  write_row_hits_0: 0
  read_row_hits_core_0: 0
  read_row_misses_core_0: 0
  num_read_reqs_0: 4
Scheduler:
  impl: FRFCFS
RefreshManager:
  impl: AllBank

RowPolicy:
  impl: ClosedRowPolicy
  num_close_reqs: 0
```

Cycle 5

- Frontend tick():
 - send(W #5)
 - write_buffer.enqueue(#5)
- Controller tick():
 - read_buf=2, write_buf=3, pending=0
 - s_read_queue_len += 2
 - s_write_queue_len += 3
 - 여기서 statistics 저장
 - 누적: s_read_queue=7, s_write_queue=9
 - Schedule: active_buffer에서 #1 선택, command=RD
 - check_ready: nRCD=16이므로,
 - clk(5) < ready_clk(16) → false → 처리 안 됨
 - 이제 read_buffer에서 command 골라야 함
 - R #2, get_preq_command() → 애도 nRCD 만족 X
 - R #4, get_preq_command() → 애도 nRCD 만족 X
 - command없기에, 이번 cycle은 command 실행 X
 - read_buf=2, active_buf=1, write_buf=3, pending=0

Example - Report의 Read/Write Buffer Length 계산되는 흐름

□ Read/Write Buffer Length 분석

- 7 Cycle동안의 동작만 분석

		example_LDST.trace
R #0	1	LD 0x00000000
W #1	2	ST 0x00000400
R #2	3	LD 0x00000800
W #3	4	ST 0x00000C00
R #4	5	LD 0x00001000
W #5	6	ST 0x00001400
R #6	7	LD 0x00001800
W #7	8	ST 0x00001C00

```
Frontend:
  impl: LoadStoreTrace

MemorySystem:
  impl: GenericDRAM
  total_num_other_requests: 0
  total_num_write_requests: 4
  total_num_read_requests: 4
  memory_system_cycles: 7
DRAM:
  impl: DDR4
AddrMapper:
  impl: RoBaRaCoCh

Controller:
  impl: Generic
  id: Channel 0
  avg_read_latency_0: 0
  read_queue_len_avg_0: 1.42857146
  write_queue_len_0: 12
  queue_len_0: 22
  num_other_reqs_0: 0
  num_write_reqs_0: 4
  read_latency_0: 0
  priority_queue_len_avg_0: 0
  row_hits_0: 0
  priority_queue_len_0: 0
  row_misses_0: 1
  row_conflicts_0: 0
  read_row_misses_0: 1
  queue_len_avg_0: 3.14285707
  read_row_conflicts_core_0: 0
  read_row_hits_0: 0
  write_queue_len_avg_0: 1.71428573
  read_row_conflicts_0: 0
  write_row_misses_0: 0
  write_row_conflicts_0: 0
  read_queue_len_0: 10
  write_row_hits_0: 0
  read_row_hits_core_0: 0
  read_row_misses_core_0: 0
  num_read_reqs_0: 4
Scheduler:
  impl: FRFCFS
RefreshManager:
  impl: AllBank

RowPolicy:
  impl: ClosedRowPolicy
  num_close_reqs: 0
```

Cycle 6

- Frontend tick():
 - send(R #6)
 - read_buffer.enqueue(#6)
- Controller tick():
 - read_buf=3, write_buf=3, pending=0
 - s_read_queue_len += 3
 - s_write_queue_len += 3
 - 여기서 statistics 저장
 - 누적: s_read_queue=10, s_write_queue=12
 - Schedule: active_buffer에서 #1 선택, command=RD
 - check_ready: nRCD=16이므로,
 - clk(6) < ready_clk(16) → false → 처리 안 됨
 - 이제 read_buffer에서 command 골라야 함
 - R #2, get_preq_command() → 애도 nRCD 만족 X
 - R #4, get_preq_command() → 애도 nRCD 만족 X
 - R #6, get_preq_command() → 애도 nRCD 만족 X
 - command없기에, 이번 cycle은 command 실행 X
 - read_buf=3, active_buf=1, write_buf=3, pending=0

Example - Report의 Read/Write Buffer Length 계산되는 흐름

□ Read/Write Buffer Length 분석

- 7 Cycle동안의 동작만 분석

		example_LDST.trace
R #0	1	LD 0x00000000
W #1	2	ST 0x00000400
R #2	3	LD 0x00000800
W #3	4	ST 0x00000C00
R #4	5	LD 0x00001000
W #5	6	ST 0x00001400
R #6	7	LD 0x00001800
W #7	8	ST 0x00001C00

```
Frontend:
  impl: LoadStoreTrace

MemorySystem:
  impl: GenericDRAM
  total_num_other_requests: 0
  total_num_write_requests: 4
  total_num_read_requests: 4
  memory_system_cycles: 7
DRAM:
  impl: DDR4
AddrMapper:
  impl: RoBaRaCoCh

Controller:
  impl: Generic
  id: Channel 0
  avg_read_latency_0: 0
  read_queue_len_avg_0: 1.42857146
  write_queue_len_0: 12
  queue_len_0: 22
  num_other_reqs_0: 0
  num_write_reqs_0: 4
  read_latency_0: 0
  priority_queue_len_avg_0: 0
  row_hits_0: 0
  priority_queue_len_0: 0
  row_misses_0: 1
  row_conflicts_0: 0
  read_row_misses_0: 1
  queue_len_avg_0: 3.14285707
  read_row_conflicts_core_0: 0
  read_row_hits_0: 0
  write_queue_len_avg_0: 1.71428573
  read_row_conflicts_0: 0
  write_row_misses_0: 0
  write_row_conflicts_0: 0
  read_queue_len_0: 10
  write_row_hits_0: 0
  read_row_hits_core_0: 0
  read_row_misses_core_0: 0
  num_read_reqs_0: 4
Scheduler:
  impl: FRFCFS
RefreshManager:
  impl: AllBank

RowPolicy:
  impl: ClosedRowPolicy
  num_close_reqs: 0
```

Cycle 7

- Frontend tick():
 - send(W #7)
 - write_buffer.enqueue(#7)
- 이제 frontend의 is_finished가 true이므로, simul 끝